NAME : DARSSHINI BABU RAJU
ROLL NO : 2025115144

1)Write a C program to construct an expression tree for a given expression.

Eg. (a-b) + (c*d)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node *stack[100];
int top = -1;

struct Node *createNode(char ch) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = ch;
    n->left = n->right = NULL;
    return n;
}

int precedence(char ch) {
    if (ch == '+' || ch == '-') return 1;
    if (ch == '*' || ch == '/') return 2;
    return 0;
}

void push(struct Node *n) {
    stack[++top] = n;
}

struct Node *pop() {
    return stack[top--];
}
```

```
void inorder(struct Node *root) {
    if (root) {
        if (root->left) printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (root->right) printf(")");
    }
}

void preorder(struct Node *root) {
    if (root) {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root) {
        postorder(root->left);
        postorder(root->right);
        printf("%c ", root->data);
    }
}

int main() {
    char exp[] = "(a-b)+(c*d)";
    char ops[100];
    int ot = -1;

    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];

        if (isalnum((unsigned char)ch)) {
            push(createNode(ch));
        } else if (ch == '(') {
            ops[++ot] = ch;
        } else if (ch == ')') {
```

```
        while (ot >= 0 && ops[ot] != '(') {
            char op = ops[ot--];
            struct Node *r = pop();
            struct Node *l = pop();
            struct Node *n = createNode(op);
            n->left = l;
            n->right = r;
            push(n);
        }
        ot--;
    } else if (ch == '+' || ch == '-' ||
               ch == '*' || ch == '/') {
        while (ot >= 0 && ops[ot] != '(' &&
               precedence(ops[ot]) >= precedence(ch)) {
            char op = ops[ot--];
            struct Node *r = pop();
            struct Node *l = pop();
            struct Node *n = createNode(op);
            n->left = l;
            n->right = r;
            push(n);
        }
        ops[++ot] = ch;
    }
}

while (ot >= 0) {
    char op = ops[ot--];
    struct Node *r = pop();
    struct Node *l = pop();
    struct Node *n = createNode(op);
    n->left = l;
    n->right = r;
    push(n);
}

struct Node *root = pop();

printf("Inorder: ");
inorder(root);
```

```
    printf("\nPreorder: ");
    preorder(root);
    printf("\nPostorder: ");
    postorder(root);

    return 0;
}
```

OUTPUT :

```
Inorder: (a-b)+(c*d)
Preorder: + - a b * c d
Postorder: a b - c d * +
```

2) Write a C program to check whether the given binary tree is a binary search tree.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *createNode(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

int isBST(struct Node *root, long min, long max) {
    if (root == NULL)
        return 1;

    if (root->data <= min || root->data >= max)
        return 0;
```

```
    return isBST(root->left, min, root->data) &&
           isBST(root->right, root->data, max);
}

int main() {
    struct Node *root = createNode(40);
    root->left = createNode(20);
    root->right = createNode(60);
    root->left->left = createNode(10);
    root->left->right = createNode(30);

    if (isBST(root, LONG_MIN, LONG_MAX))
        printf("The given binary tree is a BST.");
    else
        printf("The given binary tree is not a BST.");

    return 0;
}
```

OUTPUT :

The given binary tree is a BST.

3) Write a C program to implement a max heap.

```
#include <stdio.h>

int heap[100], n = 0;

void insert(int value) {
    int i = ++n;

    while (i > 1 && heap[i / 2] < value) {
        heap[i] = heap[i / 2];
        i = i / 2;
    }
    heap[i] = value;
}

void display() {
```

```
    for (int i = 1; i <= n; i++)
        printf("%d ", heap[i]);
    printf("\n");
}

int main() {
    insert(30);
    insert(10);
    insert(50);
    insert(20);
    insert(40);

    printf("Max Heap: ");
    display();

    return 0;
}
```

OUTPUT :
Max Heap: 50 40 30 10 20

4) Given an undirected graph the task is to check if there is a cycle in the given graph.

```
#include <stdio.h>

int graph[20][20], visited[20], n;

int dfs(int v, int parent) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i]) {
            if (!visited[i]) {
                if (dfs(i, v))
                    return 1;
            } else if (i != parent) {
                return 1;
            }
        }
    }
```

```
    return 0;
}

int main() {
    int e, u, v;

    printf("Enter number of vertices and edges: ");
    scanf("%d%d", &n, &e);

    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dfs(i, -1)) {
            printf("Cycle exists");
            return 0;
        }
    }

    printf("No cycle exists");
    return 0;
}
```

OUTPUT :

```
Enter number of vertices and edges: 4 4
0 1
1 2
2 3
3 0
Cycle exists
```

5) Given an n x m grid of 'W' (water) and 'L' (land). The task is to count number of islands

```
#include <stdio.h>

int n, m;
```

```
char grid[20][20];

void dfs(int r, int c) {
    if (r < 0 || r >= n || c < 0 || c >= m ||
        grid[r][c] != 'L')
        return;

    grid[r][c] = 'W';

    dfs(r - 1, c);
    dfs(r + 1, c);
    dfs(r, c - 1);
    dfs(r, c + 1);
}

int main() {
    printf("Enter rows and columns: ");
    scanf("%d%d", &n, &m);

    printf("Enter the grid (L/W):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf(" %c", &grid[i][j]);

    int islands = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 'L') {
                islands++;
                dfs(i, j);
            }
        }
    }

    printf("Number of islands = %d", islands);
    return 0;
}
```

OUTPUT :

```
Enter rows and columns: 4 5
Enter the grid (L/W):
L L W W L
L W W L L
W W L W W
L W W W L
Number of islands = 5
```

6) Find out the minimum spanning tree using Kruskals algorithm.

```
#include <stdio.h>

struct Edge {
    int u, v, w;
};

struct Edge e[100], temp;
int parent[100];

int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}

int main() {
    int n, m, cost = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < m; i++)
        scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

    for (int i = 0; i < n; i++)
        parent[i] = i;
```

```
    for (int i = 0; i < m - 1; i++)
        for (int j = 0; j < m - i - 1; j++)
            if (e[j].w > e[j + 1].w) {
                temp = e[j];
                e[j] = e[j + 1];
                e[j + 1] = temp;
            }

    printf("Edges in MST:\n");
    int count = 0;

    for (int i = 0; i < m && count < n - 1; i++) {
        int a = find(e[i].u);
        int b = find(e[i].v);

        if (a != b) {
            printf("%d - %d : %d\n",
                   e[i].u, e[i].v, e[i].w);
            parent[a] = b;
            cost += e[i].w;
            count++;
        }
    }

    if (count == n - 1)
        printf("Minimum cost = %d\n", cost);
    else
        printf("Spanning tree does not exist\n");

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 4 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
```

Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19

7)Use Dijkstra's algorithm to find shortest path from a source to all nodes.

```
#include <stdio.h>
#define INF 99999

int main() {
    int n, graph[20][20], dist[20], visited[20];
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &source);

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[source] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;

        for (int i = 0; i < n; i++)
            if (!visited[i] &&
                (u == -1 || dist[i] < dist[u]))
                u = i;
```

```
        if (u == -1 || dist[u] == INF)
            break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 && !visited[v] &&
                dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
        }
    }

    printf("Shortest distances from source %d:\n", source);
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("%d -> Unreachable\n", i);
        else
            printf("%d -> %d\n", i, dist[i]);
    }

    return 0;
}
```

OUTPUT :

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 4 0
1 0 2 6
4 2 0 3
0 6 3 0
Enter source vertex: 0
Shortest distances from source 0:
0 -> 0
1 -> 1
2 -> 3
3 -> 6
```

8) Write a program to find a mother vertex in the graph. A mother vertex in a graph G = (V, E) is a vertex V such that all other vertices in G can be reached by a path from V.

```c
#include <stdio.h>

int graph[20][20], visited[20], n;

void dfs(int v) {
    visited[v] = 1;

    for (int i = 0; i < n; i++)
        if (graph[v][i] && !visited[i])
            dfs(i);
}

int main() {
    int e, u, v, candidate = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    printf("Enter directed edges (u v):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            visited[j] = 0;

        dfs(i);

        int all = 1;
        for (int j = 0; j < n; j++)
            if (!visited[j])
                all = 0;

        if (all) {
            candidate = i + 1;
            break;
```

```
        }
    }

    if (candidate)
        printf("Mother vertex = %d", candidate - 1);
    else
        printf("No mother vertex exists");

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 4 4
Enter directed edges (u v):
0 1
0 2
1 2
2 3
Mother vertex = 0
```

9) A directed graph is strongly connected if there is a path between all pairs of vertices. A strongly connected component of a directed graph is a maximal strongly connected subgraph. Given a graph print all SCC.

```
#include <stdio.h>

int g[20][20], rev[20][20], visited[20];
int order[20], top = 0, n;

void dfs1(int v) {
    visited[v] = 1;
    for (int i = 0; i < n; i++)
        if (g[v][i] && !visited[i])
            dfs1(i);
    order[top++] = v;
}

void dfs2(int v) {
```

```
    visited[v] = 1;
    printf("%d ", v);
    for (int i = 0; i < n; i++)
        if (rev[v][i] && !visited[i])
            dfs2(i);
}

int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    printf("Enter directed edges (u v):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        g[u][v] = 1;
        rev[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs1(i);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("Strongly Connected Components:\n");
    while (top > 0) {
        int v = order[--top];
        if (!visited[v]) {
            dfs2(v);
            printf("\n");
        }
    }

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 5 5
Enter directed edges (u v):
0 1
1 0
1 2
2 3
3 2
Strongly Connected Components:
4
2 3
0 1
```

10) A bipartite graph G = (V, E) is a graph such that V can be partitioned into two subsets V1 and V2 and no edge has both its vertices in the same subset. Write a program to determine whether a graph is a bipartite graph.

```
#include <stdio.h>

int graph[20][20], color[20], n;

int bfs(int start) {
    int q[100], front = 0, rear = 0;

    q[rear++] = start;
    color[start] = 0;

    while (front < rear) {
        int u = q[front++];

        for (int v = 0; v < n; v++) {
            if (graph[u][v]) {
                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    q[rear++] = v;
                } else if (color[v] == color[u]) {
                    return 0;
                }
            }
```

```
        }
    }
    return 1;
}

int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    printf("Enter undirected edges (u v):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        color[i] = -1;

    for (int i = 0; i < n; i++) {
        if (color[i] == -1 && !bfs(i)) {
            printf("The graph is not bipartite");
            return 0;
        }
    }

    printf("The graph is bipartite");
    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 4 4
Enter undirected edges (u v):
0 1
1 2
2 3
3 0
The graph is bipartite
```